Studio Nx

Backend(FastAPI/Python) 57 — get_membership() · require_role()

2026-10-12

Backend(FastAPI/Python) 57 — `get_membership()` · `require_role()` (프로젝트 권한 검사)

항목	내용
파일	`studionx/backend/app/services/project_service.py`
작업	P1-13 프로젝트 API: 모든 프로젝트 API가 맨 처음 하는 검사 (PRD 01 §5 권한 규칙)
작성	Claude (AI), 2026-10-08, 사용자 검토
이전 · 다음	56 프로젝트 요청 스키마 · 58 `to_item` · `list_projects`

1. 하는 일

- `get_membership`: 프로젝트 번호로 프로젝트와 **내 멤버 행**을 함께 찾습니다. 접근할 수 없으면 404로 응답합니다.
- `require_role`: 그 멤버 행의 역할이 허용 목록에 있는지 확인합니다. 없으면 403으로 응답합니다.

상황	결과
내가 멤버인 프로젝트	`(project, member)`를 돌려줍니다
없는 번호	404 `"Project not found."`
남의 프로젝트 (내가 멤버가 아님)	404 `"Project not found."` 입니다. 403이 아닙니다 (사용자 결정 A)
삭제된 프로젝트 (`deleted_at`이 있음)	주인이어도 404입니다
VIEWER가 수정하거나 EDITOR가 삭제하려 함	403 `"You don't have permission."` (`require_role`)

2. 언제 쓰나

호출하는 곳	쓰는 것
`GET /projects/{id}`처럼 프로젝트 하나를 다루는 라우터 (62)	`get_membership`: 멤버라면 누구나 조회할 수 있습니다
`update_project` (60)	`require_role(member, "OWNER", "EDITOR")`
`delete_project` (61)	`require_role(member, "OWNER")`
시퀀서 API와 협업 API (Phase 2, 6)	같은 두 함수를 그대로 사용합니다

역할(`project_members.role`)은 OWNER, EDITOR, VIEWER 세 가지입니다. Phase 1에서는 프로젝트를 만든 사람의 OWNER 행 하나만 생깁니다 (초대 기능은 Phase 6).

3. 코드

import (파일 맨 위)

```
from sqlalchemy.orm import Session

from app.core.errors import AppError, ErrorCode
from app.models import Project, ProjectMember, User
```

```
from app.schemas.project import (
    ProjectCreateRequest,
    ProjectItem,
    ProjectListResponse,
    ProjectSort,
    ProjectUpdateRequest,
)
```

`AppError`와 `ErrorCode` 줄을 이번에 추가했습니다. 두 함수는 `Session`, `AppError`, `ErrorCode`, `Project`, `ProjectMember`, `User`를 사용합니다.

함수

```
def get_membership(db: Session, project_id: int, user: User) -> tuple[Project, ProjectMember]:
    project = db.get(Project, project_id)
    member = None
    if project is not None and project.deleted_at is None:
        member = db.get(ProjectMember, (project_id, user.user_id))
    if member is None:
        raise AppError(ErrorCode.NOT_FOUND, 404, "Project not found.")
    return project, member


def require_role(member: ProjectMember, *roles: str) -> None:
    if member.role not in roles:
        raise AppError(ErrorCode.FORBIDDEN, 403, "You don't have permission.")
```

문법 풀이

import 문법	뜻
`from app.core.errors import AppError, ErrorCode`	이 프로젝트의 오류 클래스와 오류 코드 목록입니다 (P1-03). `AppError`를 던지면 처리기가 `{"error": {...}}` 형태의 JSON으로 바꿉니다

식	뜻
`db.get(Project, project_id)`	기본 키로 행 하나를 찾습니다. 없으면 `None`입니다
`db.get(ProjectMember, (project_id, user.user_id))`	기본 키가 두 칸으로 된 복합 키라면 **튜플**로 넘깁니다
`-> tuple[Project, ProjectMember]`	두 값을 함께 돌려준다는 표시입니다. 부르는 쪽은 `project, member = get_membership(...)`로 받습니다
`*roles: str`	인자를 몇 개든 받아서 튜플로 모읍니다. `require_role(m, "OWNER", "EDITOR")`라면 `roles == ("OWNER", "EDITOR")`입니다
`member.role not in roles`	역할은 `MemberRole`(StrEnum)이라서 문자열 `"OWNER"`와 같은 값으로 비교됩니다

4. 흐름

1. 번호로 프로젝트를 찾습니다.
2. 프로젝트가 있고 삭제되지 않았을 때만 `(프로젝트 번호, 내 회원 번호)`로 멤버 행을 찾습니다.
3. 멤버 행이 없으면, 그 이유(번호 없음, 삭제됨, 남의 프로젝트)와 상관없이 404로 응답합니다.
4. 수정이나 삭제일 때는 `require_role`로 역할을 확인하고, 모자라면 403으로 응답합니다.

5. 왜 이렇게

선택	이유
남의 프로젝트도 **404**	403으로 응답하면 "그 번호의 프로젝트는 존재한다"는 사실이 드러납니다. 번호를 1, 2, 3... 차례로 넣어 보며 남의 프로젝트가 있는지 알아낼 수 없게 했습니다 (사용자 결정 A, 템플릿 권장)
세 경우를 `member is None` 하나로 처리	번호 없음, 삭제됨, 남의 프로젝트가 모두 같은 응답이어야 하므로 오류를 한 곳에서만 던집니다
삭제된 프로젝트는 멤버 행을 찾지 않음	불필요한 쿼리를 줄이고, 주인이어도 삭제된 프로젝트는 볼 수 없게 합니다 (소프트 삭제, 61)
역할이 부족하면 **403**	이미 멤버라서 프로젝트가 있다는 것을 아는 사람입니다. 숨길 것이 없으므로 "권한 없음"이 더 정확한 응답입니다
두 함수로 나눔	조회는 멤버이기만 하면 충분하고, 수정과 삭제만 역할을 확인합니다. 그래서 필요한 곳에서만 `require_role`을 한 줄 더 부릅니다

6. 시험 (`tests/test_project_service.py` 새 파일, 12개)

시험	확인한 것
주인	`(project, member)`를 돌려주고, 역할은 OWNER입니다
EDITOR 멤버	멤버 행을 찾고, 역할은 EDITOR입니다
없는 번호	404 `NOT_FOUND` "Project not found."
남의 프로젝트	404입니다 (403이 아닙니다)
삭제된 프로젝트	주인이어도 404입니다
`require_role` 통과 4가지	OWNER만 허용할 때 OWNER, OWNER·EDITOR를 허용할 때 OWNER와 EDITOR, 셋 다 허용할 때 VIEWER
`require_role` 거부 3가지	OWNER만 허용할 때 EDITOR와 VIEWER, OWNER·EDITOR를 허용할 때 VIEWER는 403 `FORBIDDEN`입니다

전체 시험 450개가 통과했고, `ruff` 검사도 통과했습니다.